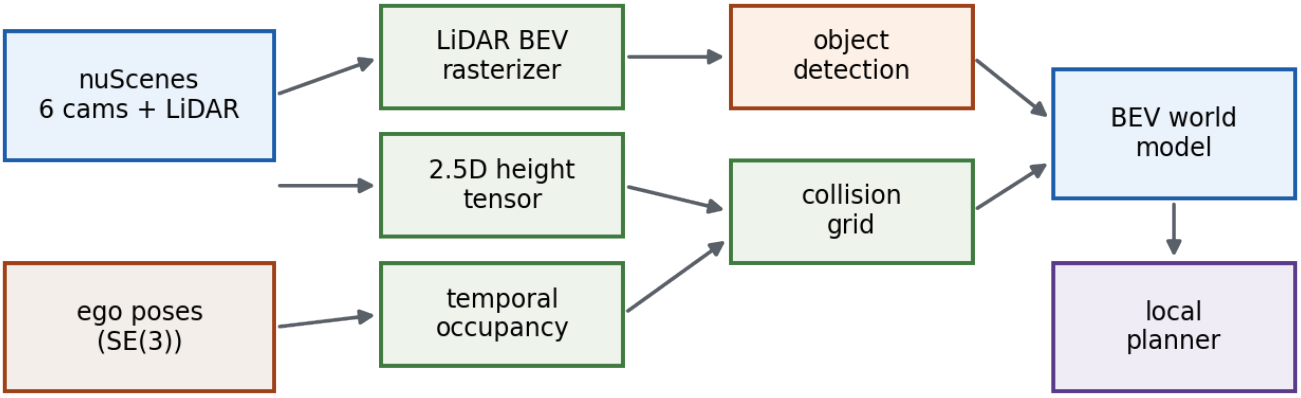
TECHNICAL WHITEPAPER

A LiDAR-Camera Bird's-Eye-View World Model on nuScenes

The 360-degree surround-vision arm of the vision-fsd project: geometry, math, and implementation from raw sensors to a stateful BEV world model and a classical local planner.



Scope: the nuScenes 360 pipeline only (fsd/). The monocular dashcam stack is deliberately excluded. Every section maps to source in fsd/ and is written to be reproducible from the code.

SECTION 1

System Overview

The 360 pipeline turns one nuScenes keyframe - six surround cameras, one 32-beam LiDAR sweep, and the vehicle's ego pose - into a **bird's-eye-view (BEV) world model**: a top-down, metric, ego-centred description of the scene that a planner can reason over. The defining design move is the shift from a **frame-centric** system (process a frame, draw it, discard it) to a **stateful** one that warps and accumulates evidence across time.

Everything is built on three coordinate frames and the rigid transforms between them, so Section 2 establishes that machinery first. The rest of the document then follows the data: LiDAR geometry and BEV rasterization (3), the 2.5D height tensor (4), temporal occupancy fusion (5), 3D object detection by three independent routes (6), the camera-only Lift-Splat-Shoot BEV (7), the unified world model (8), and the classical local planner (9).

Module map

Each capability is one module under `fsd/`. The visualizer (`fsd/visualize.py`) selects a *view* and drives the per-frame render loop.

```
fsd/data.py              read-only nuScenes loader (6 cams + LiDAR, lazy)
fsd/lidar_projection.py  quaternion/rigid transforms, LiDAR->camera, pinhole
fsd/bev.py               ego-frame LiDAR BEV rasterizer
fsd/bev_tensor.py        2.5D per-cell height-channel tensor
fsd/occupancy.py         temporal log-odds occupancy fusion
fsd/object_detection.py  GT 3D boxes + prediction-JSON adapter + BEV drawing
fsd/fusion_detect.py     camera(YOLO)+LiDAR frustum-fusion detector
fsd/centerpoint_export.py  pretrained CenterPoint (isolated mmdet3d env)
fsd/lss.py               Lift-Splat-Shoot camera-only BEV segmentation
fsd/world_model.py       unified BevWorldModel
fsd/motion_planning/     ego state, lattice sampler, validator, costs, planner
```

SECTION 2

Data and Coordinate Frames

nuScenes is ~850 independent 20-second clips (*scenes*). Each scene is a chain of *samples* (keyframes) at 2 Hz, plus higher-rate intermediate *sweeps*. A sample links to one `sample_data` record per sensor channel, and each record carries a `calibrated_sensor` (sensor->ego extrinsic + camera intrinsic) and an `ego_pose` (ego->global pose at that timestamp).

2.1 Read-only streaming loader

The dataset is treated as external and read-only: nothing is copied into the repo, and there is **no nuscenes-devkit runtime dependency** in `fsd/`. Metadata is read straight from the JSON tables. The large tables (`sample_data.json`, `ego_pose.json`, `sample_annotation.json`) are hundreds of MB, so `data.py` streams them object by object instead of loading the whole array, stopping as soon as the requested tokens are found.

```
def _iter_json_objects(path):
    with path.open('r', encoding='utf-8') as handle:
        collecting, depth, lines = False, 0, []
        for raw_line in handle:
            stripped = raw_line.strip()
            if not collecting:
                if stripped.startswith('{'):
                    collecting = True; lines = [raw_line]
                    depth = raw_line.count('{') - raw_line.count('}')
                continue
            lines.append(raw_line)
            depth += raw_line.count('{') - raw_line.count('}')
            if depth == 0:
                text = ''.join(lines).strip().rstrip(',')
                yield json.loads(text); collecting = False; lines = []
```

fsd/data.py - brace-counting streaming parser; tokens are cached on first hit.

Frames are exposed as immutable dataclasses: `CameraFrame` (image path + intrinsic + extrinsic + ego pose), `LidarFrame` (point-cloud path + extrinsic + ego pose), and `SurroundFrame` bundling the six cameras for one sample. Image and point-cloud files are loaded lazily, only when a view actually needs them.

2.2 The three frames

- **Sensor frame** - raw measurements (LiDAR points, camera rays) in the sensor's own axes.
- **Ego frame** - the vehicle body frame at one timestamp: x forward, y left, z up. The world model lives here.
- **Global frame** - a fixed map frame per log. Ego motion across time is expressed here, which is what makes temporal fusion possible.

Every transform is rigid: a rotation R and a translation t. nuScenes stores rotations as unit quaternions in **w, x, y, z** order. The conversion to a 3x3 matrix (after normalising q):

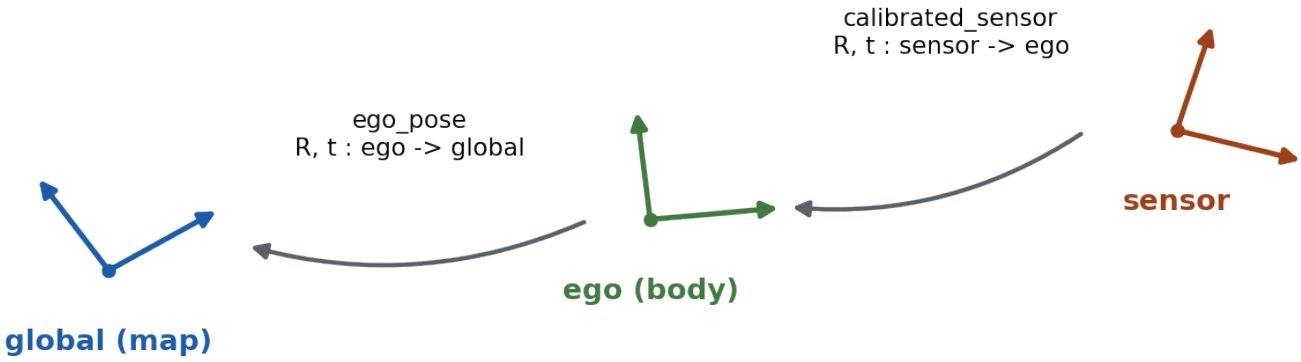
```
w, x, y, z = q / |q|
R = | 1-2(y2+z2)   2(xy-zw)    2(xz+yw) |
    | 2(xy+zw)     1-2(x2+z2)  2(yz-xw) |
    | 2(xz-yw)     2(yz+xw)    1-2(x2+y2) |
```

fsd/lidar_projection.py - quaternion_to_rotation_matrix (x2 == x squared).

A source->target transform and its exact inverse are the two workhorses of the whole codebase (points are stored as row vectors, hence the transpose):

$$\mathbf{p}_{\text{tgt}} = \mathbf{p}_{\text{src}} R^{\top} + \mathbf{t} \qquad \Longleftrightarrow \qquad \mathbf{p}_{\text{src}} = (\mathbf{p}_{\text{tgt}} - \mathbf{t})\, R \tag{1}$$

These two functions - `transform_points` and `inverse_transform_points` - compose into every sensor->ego->global->sensor chain that follows.



cameras invert this chain (global -> ego -> sensor) to project into the image

Figure 1. The three coordinate frames and the rigid transforms between them. LiDAR points flow sensor -> ego -> global; cameras invert the chain.

SECTION 3

LiDAR Geometry: Projection and BEV

A nuScenes LIDAR_TOP sweep is a binary blob of 5 floats per point (x, y, z, intensity, ring). Only XYZ is used for geometry; the cloud is reshaped to Nx5 and sliced to Nx3.

3.1 LiDAR into the cameras

To colour a camera image by LiDAR depth, sensor-frame points are walked through the full chain into the camera frame: LiDAR-sensor -> ego(LiDAR time) -> global -> ego(camera time) -> camera-sensor. The first two hops are forward transforms; the last two are inverse transforms.

```
points = transform_points(lidar_pts, lidar.cs.rot, lidar.cs.trans)      # -> ego
points = transform_points(points,    lidar.ego.rot, lidar.ego.trans)    # -> global
points = inverse_transform_points(points, cam.ego.rot, cam.ego.trans)   # -> ego(cam)
points = inverse_transform_points(points, cam.cs.rot,  cam.cs.trans)    # -> camera
```

fsd/lidar_projection.py - lidar_points_to_camera.

Camera-frame points are then projected to pixels with the pinhole intrinsic K. Points behind the image plane (z <= min_depth) are dropped first:

$$\mathbf{u} = \frac{1}{z} K \mathbf{p}_{\text{cam}}, \qquad u = f_x \frac{x}{z} + c_x, \quad v = f_y \frac{y}{z} + c_y \tag{2}$$

Depth is mapped through a TURBO colormap (near = warm) so the projected points read as a depth image overlaid on each of the six cameras.

3.2 Ego-frame BEV rasterization

The BEV is a top-down grid over a metric window - default x,y in [-50, 50] m at 0.25 m/cell, giving a 400x400 grid. Sensor points are first moved into the ego frame, then each surviving point's metric (x, y) maps to an integer (row, col). Forward (+x) is up, left (+y) is left, so:

$$\text{row} = \left\lfloor \frac{x_{\max} - x}{\Delta} \right\rfloor, \qquad \text{col} = \left\lfloor \frac{y_{\max} - y}{\Delta} \right\rfloor \tag{3}$$

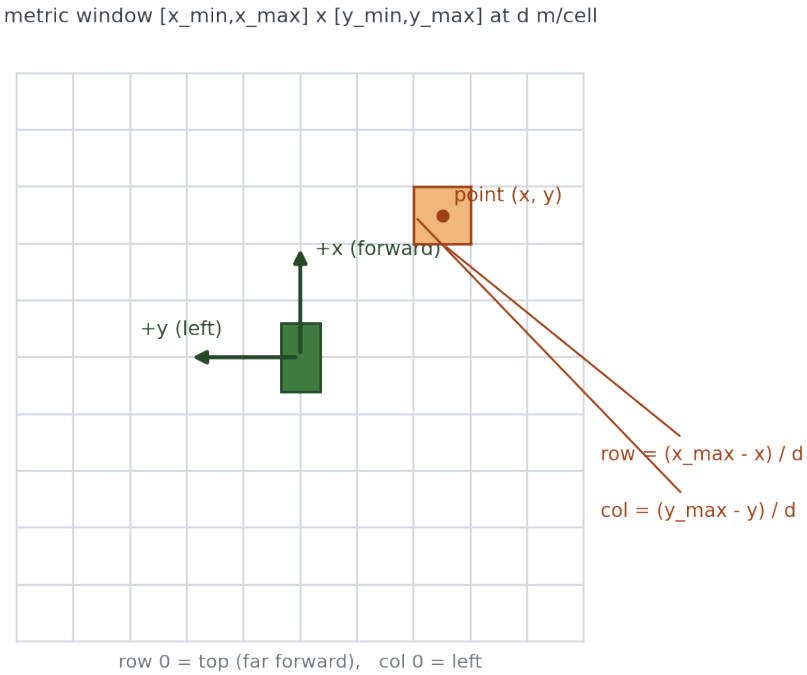


Figure 2. Ego-frame BEV rasterization. Each metric (x, y) maps to an integer (row, col); forward is up, left is left. The same map is reused by every BEV layer so they overlay cleanly.

with grid resolution **Delta** (metres per cell). Points outside the window or outside a z-band are masked out; the rest are coloured by height (TURBO) and written to the canvas. This same metric->pixel map, with the same origin convention, is reused by the height tensor, the occupancy grid, and box drawing - consistency here is what lets the layers overlay cleanly.

SECTION 4

2.5D BEV Height-Channel Tensor

A plain BEV only records 'cell has points or not'. The 2.5D tensor keeps the vertical structure that a single occupancy bit throws away. Every channel is derived from the LiDAR sweep alone - no labels, no map - so it is deployable on a real vehicle. It is the data structure downstream modules read instead of re-rasterizing raw points.

Per-cell channels

- **density** - point count in the cell (surface support / confidence).
- **max_height / min_height / mean_height** - vertical extremes and average of returns in the cell.
- **height_range** = max_height - min_height - the discriminative one.

The key insight: **height_range** separates drivable surface from obstacles with pure geometry. Flat road produces returns at nearly one height, so its range is ~0; a car or wall spans a large vertical extent, so its range is large. No learning required.

$$h_{\mathrm{rng}}(c) = \max_{i \in c} z_i - \min_{i \in c} z_i , \qquad \bar{z}(c) = \frac{1}{n_c} \sum_{i \in c} z_i \tag{4}$$

Rasterization is fully vectorized. Each point's (row, col) is flattened to a linear cell index, and unbuffered scatter-reductions accumulate the statistics in one pass over the cloud:

```
flat = rows * w + cols
count = np.zeros(h*w); np.add.at(count, flat, 1.0)          # density
zsum  = np.zeros(h*w); np.add.at(zsum,  flat, z)            # for mean
zmax  = np.full(h*w, -inf); np.maximum.at(zmax, flat, z)    # max height
zmin  = np.full(h*w,  inf); np.minimum.at(zmin, flat, z)    # min height
occupied = count > 0
mean = where(occupied, zsum / maximum(count, 1), 0.0)
height_range = where(occupied, zmax - zmin, 0.0)
```

fsd/bev_tensor.py - compute_bev_height_channels. np.add.at / maximum.at are the unbuffered scatter ops that make repeated cell hits accumulate correctly.

`BevTensor.stack()` returns an (H, W, 5) float array in channel order. Verified synthetically: a 2.5 m wall column reads ~2.5 m height_range; flat ground reads ~0.

SECTION 5

Temporal Occupancy Fusion

This is the step that makes the system stateful. A rolling occupancy grid is kept in **log-odds** in the current ego frame. Each keyframe the previous grid is warped into the new ego frame, decayed toward 'unknown', and updated with fresh LiDAR evidence. The result is a continuously evolving map of free and occupied space rather than a single-frame snapshot.

5.1 Log-odds occupancy

Each cell stores the log-odds of being occupied. Evidence is additive in log-odds (a Bayesian binary filter / inverse-sensor model), which is why the update is a simple add-and-clamp. The probability is recovered with the logistic sigmoid:

$$l_t = \mathrm{clip}\big(\gamma\, l_{t-1}^{\mathrm{warp}} + \Delta l,\ -L,\ +L\big), \qquad p = \sigma(l) = \frac{1}{1+e^{-l}} \tag{5}$$

A hit adds +logit_hit (0.85), a miss adds -logit_miss (0.4), the decay factor is gamma = 0.97, and log-odds are clamped to +/-5. Decay continually pulls unobserved cells back toward p = 0.5 (unknown), so stale evidence fades.

5.2 Warping the prior into the current frame

Between two keyframes the ego moves. To keep the grid ego-centred, the previous grid must be resampled into the new frame. The relative planar motion is an SE(2) built from the two global ego poses - it maps a point expressed in the *current* ego frame back to the *previous* ego frame:

$$R_{\mathrm{rel}} = R_{\mathrm{prev}}^{\top} R_{\mathrm{cur}}, \qquad \mathbf{t}_{\mathrm{rel}} = R_{\mathrm{prev}}^{\top}(\mathbf{t}_{\mathrm{cur}} - \mathbf{t}_{\mathrm{prev}}) \tag{6}$$

Because the grid is an image, this metric SE(2) is sandwiched between a pixel->metre map and a metre->pixel map to get a single 2x3 affine that `cv2.warpAffine` can apply:

$$M = M_{\mathrm{m2p}} \cdot T_{\mathrm{rel}}^{\mathrm{SE(2)}} \cdot M_{\mathrm{p2m}} \tag{7}$$

```
se2 = [[R_rel, t_rel], [0, 0, 1]]
p2m = [[0, -res,  x_max], [-res, 0, y_max], [0,0,1]]   # pixel -> metre
m2p = [[0, -1/res, y_max/res], [-1/res, 0, x_max/res], [0,0,1]]  # metre -> pixel
M = (m2p @ se2 @ p2m)[:2]
logodds = cv2.warpAffine(logodds, M, (w, h),
          flags=cv2.INTER_LINEAR | cv2.WARP_INVERSE_MAP,  # M already maps cur->prev
          borderValue=0.0)   # outside = unknown
```

fsd/occupancy.py - WARP_INVERSE_MAP is essential: M already maps current-frame pixels to previous-frame pixels, and the flag stops OpenCV inverting it again.

5.3 Evidence: a height split, not ray casting

Fresh evidence comes from a deliberately simple rule. Points are rasterized to cells; returns below ground_height (0.3 m in the ego frame) mark their cell **free**, taller returns mark it **occupied**. Occupied wins ties.

```
ground   = inbounds & (z >= z_min) & (z <  ground_height)   # road surface -> free
obstacle = inbounds & (z >= ground_height) & (z <= z_max)   # tall -> occupied
free[rows[ground],  cols[ground]]  = True
occ [rows[obstacle], cols[obstacle]] = True
free[occ] = False
logodds[free] += -logit_miss
logodds[occ]  += +logit_hit
```

fsd/occupancy.py - _evidence + step.

Why not ray-cast free space? An earlier polar 'nearest-hit' free-space polygon was dropped: ground returns are the nearest hit in nearly every direction, so casting free space up to the first hit both collapsed the free region and painted the road as occupied. Marking only observed cells, split by height, keeps the road out of the occupied set. Verified synthetically: 2 m forward motion moves a 20 m wall to 18 m; a +90 deg yaw moves a point straight ahead to the right.

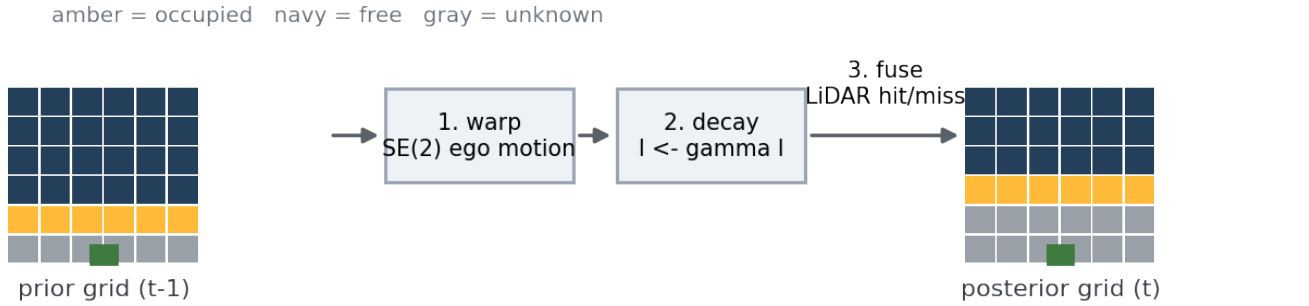


Figure 3. One temporal occupancy update: warp the prior grid into the new ego frame, decay it toward unknown, then fuse fresh LiDAR hit/miss evidence.

SECTION 6

3D Object Detection

Objects enter the world model as ego-frame `Box3D` records - center, size (w, l, h), yaw, and 2D/3D corners. Three independent routes produce them.

6.1 Ground-truth boxes from annotations

nuScenes annotations are 3D boxes in the global frame (translation, size, rotation quaternion). Each is converted to the ego frame. The four bottom corners are built in the box's local frame, rotated into global, then inverse-transformed by the ego pose; the box's ego-frame yaw comes from the composed rotation:

$$R_{\text{box}}^{\text{ego}} = R_{\text{ego}}^{\top} R_{\text{box}}^{\text{global}}, \qquad \psi_{\text{ego}} = \operatorname{atan2}(R_{10},\, R_{00}) \tag{8}$$

Boxes are filtered by `num_lidar_pts >= min_lidar_points` so empty annotations are skipped, and the nuScenes category strings are folded into a small class set (car, truck, bus, trailer, motorcycle, bicycle, pedestrian, ...).

6.2 CenterPoint - a real learned LiDAR detector

The canonical path is pretrained **CenterPoint** (mmdet3d). The OpenMMLab stack would not build on the main environment (Py3.12 / Torch 2.7 / CUDA 12.8, no prebuilt mmcv), so it lives in a separate pinned env (`.venv-mmdet3d`: Py3.11, Torch 2.1+cu121, mmcv 2.1, mmdet3d 1.4) and feeds boxes back through a prediction-JSON adapter.

The multi-sweep fix

nuScenes CenterPoint is trained on **10 accumulated LiDAR sweeps** with the per-point 5th channel set to a time delta. Naively running inference on a single .pcd.bin feeds the model ~25k points instead of ~270k (the config's multi-sweep loader just pads by duplication), which visibly hurt recall and produced spurious boxes. The exporter aggregates the real 10 sweeps itself - transforming each older sweep into the current LiDAR frame and stamping its time delta - then strips the pipeline's `LoadPointsFromMultiSweeps` so it is not re-padded.

```
ref_from_car  = T(ref_cs, inverse=True)      # current LiDAR sensor <- ego
car_from_global = T(ref_pose, inverse=True)  # ego <- global
for each older sweep sd:
    global_from_car = T(cur_pose)            # global <- ego(sd)
    car_from_current = T(cur_cs)             # ego(sd) <- sensor(sd)
    pc.transform(ref_from_car @ car_from_global @ global_from_car @ car_from_current)
    time = (ref_time - sd_time) * ones(N)    # per-point time channel
    points = concat(points, [pc.xyz, intensity, time])
```

fsd/centerpoint_export.py - _aggregate_sweeps. Effect on scene 0: ~25k -> ~270k pts/frame, fewer false positives, predictions sit tightly on GT.

CenterPoint's boxes come out in the LiDAR sensor frame; the exporter converts each center and yaw to the ego frame with the LiDAR extrinsic and writes {samples: {sample_token: [{center_ego, yaw, size, detection_name, detection_score}]}} for the visualizer.

6.3 Frustum-fusion detector - camera detects, LiDAR ranges

A second, label-free detector runs entirely on the main environment. Cameras answer *what and where in the image* (YOLO 2D boxes + COCO class); LiDAR answers *how far and how big*. It is 'frustum' detection without a learned 3D head: the 2D box defines a viewing frustum, and the LiDAR points falling

inside it localise the object in 3D.

Per camera, per 2D detection:

- Project the LiDAR sweep into the camera; keep the points whose pixels land inside the 2D box (the frustum).
- Drop ground (z <= 0.3 m). A 2D box also catches background seen around the object, so keep only the **nearest cluster**: points within a depth band of a robust near range (the 15th percentile of range).

$$r_i = \sqrt{x_i^2 + y_i^2}, \qquad \text{keep } r_i \le \mathrm{P}_{15}(r) + \text{band} \tag{9}$$

- Position = cluster XY mean; the box is rested on the ground plane at z = h/2 from a class size prior.
- Heading from the cluster's dominant horizontal axis via PCA - the eigenvector of the 2x2 scatter matrix with the largest eigenvalue (radial fallback for tiny clusters):

$$C = X^\top X, \quad C\,\mathbf{e}_k = \lambda_k \mathbf{e}_k, \quad \psi = \operatorname{atan2}(\mathbf{e}_{\max,y}, \mathbf{e}_{\max,x}) \tag{10}$$

Finally, objects seen by two overlapping cameras are de-duplicated greedily by class and centre distance (highest score wins). Honest limits: positions are as good as the LiDAR cluster (solid), but class is YOLO/COCO only, size is a prior not a measurement, and heading from a partial-view cluster is rough - good enough to feed tracking and velocity next.

SECTION 7

Camera-only BEV: Lift-Splat-Shoot

Lift-Splat-Shoot (LSS, ECCV 2020) is the first learned BEV head in the project: a camera-only model that predicts a top-down vehicle-segmentation map from the six images alone. The model code is a 1:1 port of the NVIDIA repo; the inference wrapper consumes our `SurroundFrame` directly instead of the devkit dataloader.

Lift

Each camera image is encoded (EfficientNet-B0 trunk) into, per downsampled pixel, a categorical **depth distribution** over D discrete depth bins (softmax) and a C-dim context feature. The outer product of the two 'lifts' the 2D feature into a frustum of 3D features - feature times depth-probability at every (pixel, depth):

$$\alpha \in \Delta^{D-1} \text{ (softmax depth)}, \qquad \mathbf{c}_d = \alpha_d \mathbf{f} \in \mathbb{R}^C \tag{11}$$

```
x = self.depthnet(features)                      # D + C channels
depth = x[:, :D].softmax(dim=1)                  # depth distribution
new_x = depth.unsqueeze(1) * x[:, D:D+C].unsqueeze(2)   # outer product -> (C, D, ...)
```

fsd/lss.py - CamEncode.get_depth_feat.

Splat

Each frustum point is placed in 3D using the camera intrinsic and extrinsic (`get_geometry`), then dropped into a fixed BEV voxel grid. Many frustum points fall in the same voxel, so features are **sum-pooled** per voxel. The efficiency trick: sort points by voxel rank and use a cumulative-sum so each voxel's total is one subtraction of cumsum endpoints - the autograd-friendly `QuickCumsum`.

```
ranks = geom[:,0]*(nx1*nx2*B) + geom[:,1]*(nx2*B) + geom[:,2]*B + geom[:,3]
x, geom, ranks = sort_by(ranks)
x, geom = QuickCumsum.apply(x, geom, ranks)   # sum features sharing a voxel
final[geom...] = x                            # scatter into BEV grid
```

fsd/lss.py - voxel_pooling.

Shoot (not in the port)

The released LSS code ships only the lift+splat vehicle-seg head. The cost-map / template-trajectory 'shoot' stage from the paper is not in the open-source repo, so it is not in this port. The wrapper replicates the eval-time resize+center-crop (to 128x352) and the six per-camera intrinsics/extrinsics, runs the pretrained checkpoint, and returns a 200x200 @ 0.5 m/cell probability grid reoriented to match the LiDAR BEV. An HD-map backdrop (parsed directly from the nuScenes map-expansion vectors) can be drawn behind it.

SECTION 8

The Unified BEV World Model

`WorldModelBuilder.step()` composes the static and dynamic layers into one immutable `BevWorldModel`: temporal occupancy probability, the 2.5D height tensor, a derived collision grid, ego state, and object footprints (GT and/or prediction boxes). It is the handoff point between perception and planning.

Collision grid = occupancy OR height

The single most planning-relevant derived layer fuses the occupancy probability with the height tensor. A cell is blocked if it is probably occupied **or** tall - the height channel catches obstacles the occupancy filter has not yet committed to, and vice versa:

$$\text{blocked}(c) = [p(c) \geq \tau_p] \;\vee\; [h_{\text{rng}}(c) \geq \tau_h] \tag{12}$$

```
blocked = (occupancy_probability >= 0.62) | (height_range >= 0.45)
```

fsd/motion_planning/occupancy.py - build_collision_grid (tau_p=0.62, tau_h=0.45 m).

The objects are still recomputed each frame - they are an *overlay*, not yet part of the model's memory (no track id, velocity, or history). Closing that gap (tracking -> velocity -> short-horizon prediction) is the next milestone.

SECTION 9

Classical Local Planner

The planner is deliberately classical and debuggable: estimate ego state, sample a lattice of timed trajectories, reject colliding ones against the collision grid, score the survivors, and pick the cheapest - with an emergency stop as fallback.

9.1 Ego state from poses

Speed and yaw-rate are finite-differenced from consecutive ego poses (first frame falls back to zero):

$$v = \frac{\sqrt{\Delta x^2 + \Delta y^2}}{\Delta t}, \qquad \dot{\psi} = \frac{\mathrm{wrap}(\psi_t - \psi_{t-1})}{\Delta t} \tag{13}$$

9.2 Timed lattice of trajectories

Candidates are the cross product of target speeds {0, 2.5, 5, 7.5} m/s and curvatures {+/-0.12, +/-0.06, +/-0.03, 0} 1/m, integrated over a 3 s horizon at 0.25 s steps in the ego frame (start at origin, heading 0). Speed ramps linearly from the current speed to the target; each step advances along a **constant-curvature arc**:

$$v(t) = v_0 + (v_{\mathrm{tgt}} - v_0)\,\frac{t}{T}, \qquad \Delta\psi = \kappa\, d, \quad d = \bar{v}\,\Delta t \tag{14}$$

$$x' = x + \frac{\sin(\psi + \Delta\psi) - \sin\psi}{\kappa}, \qquad y' = y + \frac{\cos\psi - \cos(\psi + \Delta\psi)}{\kappa} \tag{15}$$

with the straight-line limit (x' = x + d cos psi, y' = y + d sin psi) used when |kappa| is ~0 to avoid dividing by zero.

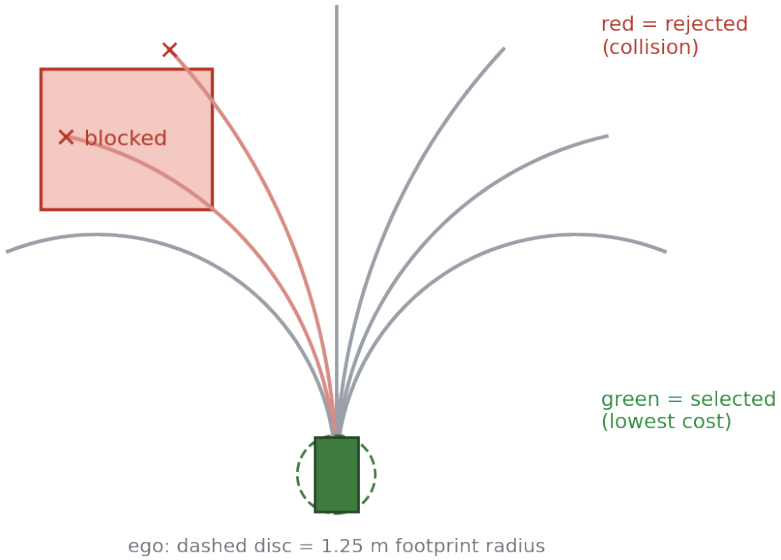


Figure 4. The timed lattice. Constant-curvature arcs fan out from the ego; arcs whose swept disc hits a blocked cell are rejected (red), and the lowest-cost survivor is selected (green).

9.3 Collision validation

Every trajectory point is checked as a **disc** of radius 1.25 m against the collision grid, and consecutive points are sub-sampled finely enough that the disc cannot tunnel through a thin obstacle between samples (spacing = min(cell/2, radius/2)). A footprint is blocked if any blocked cell lies within the radius, computed

against the closest point of each candidate cell (a proper circle-vs-cell test, not just the centre). Leaving the grid counts as blocked.

```
for step in 1..steps:                     # steps = ceil(seg_len / spacing)
    p = lerp(start, end, step/steps)
    if collision_grid.footprint_blocked(p.x, p.y, radius): return COLLISION
# footprint_blocked: any blocked cell whose closest point is within radius
```

fsd/motion_planning/validator.py - swept-disc collision check.

9.4 Cost and selection

Valid trajectories are scored by a weighted sum that rewards progress and penalises lateral offset, curvature, and speed error (a lane-centering term activates when a confident lane context is present, which is not wired yet):

$$J = -w_p L \; + \; w_l \left|y_N\right| \; + \; w_\kappa \left|\kappa\right| \; + \; w_v \left|v_{\text{tgt}} - v_0\right| \tag{16}$$

with weights (progress 4.0, lateral 1.2, curvature 2.0, speed 0.4). The lowest-cost valid trajectory is selected. If **no** candidate is collision-free, the planner returns a decelerating emergency stop (straight-ahead, ramping speed to zero) and flags whether even that is collision-free. The `OfflinePlanningRuntime` ties it together per frame - occupancy + height -> collision grid, ego state, plan - and resets its temporal state at every scene boundary.

```
occupancy   = mapper.step(lidar)                    # temporal log-odds
height      = bev_tensor_from_lidar(lidar)          # 2.5D channels
collision   = build_collision_grid(occupancy, height.height_range, ...)
ego         = estimate_ego_state(pose, t, prev_pose, prev_t)
world       = PlannerWorld(ego, collision.blocked, occupancy, ...)
result      = planner.plan(world)                   # sample -> validate -> score
```

fsd/motion_planning/runner.py - OfflinePlanningRuntime.step.

SECTION 10

Status and Next Steps

What exists today is a complete static world model plus a working object overlay and a first classical planner:

- **Stateful** temporal occupancy, 2.5D height tensor, and a fused collision grid.
- Three object-detection routes: GT boxes, pretrained CenterPoint, and the label-free camera+LiDAR frustum-fusion detector.
- Camera-only LSS BEV with an HD-map backdrop, and the unified `BevWorldModel`.
- A lattice local planner over the collision grid with swept-disc validation, cost ranking, and an emergency-stop fallback.

The clear gap is **time-aware dynamic objects**. The model sees the present: it can say 'there is a car here', but not 'that car is moving at 6 m/s and will cross the ego path in 2 s'. The next milestone gives objects identity and motion - finite-difference velocity (GT instance tokens first as an oracle, then association over CenterPoint predictions), constant-velocity short-horizon prediction, and object-aware collision risk - after which the planner can reason about other agents, not just static free space.